

```
static ssize_t my_write(struct file *f, const char __user *buf,
size_t len, loff_t *off)
{
    printk(KERN_INFO "Driver: write()\n");
    return len;
}
```

Based on the earlier understanding of the return value of the functions in the kernel, *my_open()* and *my_close()* are trivial, their return types being int, and both of them returning zero, means success. However, the return types of both *my_read()* and *my_write()* are not int, rather, it is *ssize_t*. On further digging through kernel headers, that turns out to be a signed word. So, returning a negative number would be a usual error. But a non-negative return value would have additional meaning. For the read operation, it would be the number of bytes read, and for the write operation, it would be the number of bytes written.

Reading the device file

To understand this in detail, the complete flow has to be given a relook. Let's take the read operation first. When the user does a read from the device file *'/dev/mynull'*, that system call comes to the virtual file system (VFS) layer in the kernel. VFS decodes the *<major, minor>* tuple, and figures out that it needs to redirect it to the driver's function *my_read()*, that's registered with it. So from that angle, *my_read()* is invoked as a request to read, from us—the device-driver writers. And hence, its return value would indicate to the requesters (i.e., the users), how many bytes they are getting from the read request. In our null driver example, we returned zero—which meant no bytes available, or in other words, the end of the file. And hence, when the device file is being read, the result is always nothing, independent of what is written into it.

"Hmmm...So, if I change it to 1, would it start giving me some data?" asked Pugs, by way of verifying. Shweta paused for a while, looked at the parameters of the function *my_read()* and answered in the affirmative, but with a caveat—the data sent would be some junk data, since *my_read()* is not really populating data into *buf* (the buffer variable that is the second parameter of *my_read()*, provided by the user). In fact, *my_read()* should write data into *buf*, according to *len* (the third parameter to the function), the count in bytes requested by the user.

To be more specific, it should write less than, or equal to, *len* bytes of data into *buf*, and the number of bytes written should be passed back as the return value. No, this is not a typo—in the read operation, device-driver writers 'write' into the user-supplied buffer. We read the data from (possibly) an underlying

device, and then write that data into the user buffer, so that the user can read it. "That's really smart of you," said Pugs, sarcastically.

Writing into the device file

The write operation is the reverse. The user provides *len* (the third parameter of *my_write()*) bytes of data to be written, in *buf* (the second parameter of *my_write()*). The *my_write()* function would read that data and possibly write it to an underlying device, and return the number of bytes that have been successfully written. "Aha!! That's why all my writes into *'/dev/mynull'* have been successful, without actually doing any read or write," exclaimed Shweta, filled with happiness at understanding the complete flow of device file operations.

Preserving the last character

With Shweta not giving Pugs any chance to correct her, he came up with a challenge. "Okay. Seems like you are thoroughly clear with the read/write fundamentals; so, here's a question for you. Can you modify these *my_read()* and *my_write()* functions such that whenever I read */dev/mynull*, I get the last character written into */dev/mynull*?"

Confidently, Shweta took on the challenge, and modified *my_read()* and *my_write()* as follows, adding a static global character variable:

```
static char c;

static ssize_t my_read(struct file *f, char __user *buf,
size_t len, loff_t *off)
{
    printk(KERN_INFO "Driver: read()\n");
    buf[0] = c;
    return 1;
}

static ssize_t my_write(struct file *f, const char __user
*buf, size_t len, loff_t *off)
{
    printk(KERN_INFO "Driver: write()\n");
    c = buf[len - 1];
    return len;
}
```

"Almost there, but what if the user has provided an invalid buffer, or if the user buffer is swapped out. Wouldn't this direct access of the user-space *buf* just crash and oops the kernel?" pounced Pugs. Shweta, refusing to be intimidated, dived into her collated material and figured out that there are two APIs just to ensure that user-space buffers are safe to access, and then updated them. With the complete understanding of the APIs, she re-wrote the above code snippet as follows: